

inplace_vector

A dynamically-resizable vector with fixed capacity and embedded storage

Document number: P0843R7.

Date: 2023-06-16.

Authors: Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel <dsankel _at_ adobe.com (<http://adobe.com>)>.

Reply to: Timur Doumler <papers _at_ timur.audio>.

Audience: LEWG.

Table of Contents

- `inplace_vector`
- Introduction
- Motivation and Scope
- Existing practice
- Design
 - Standalone or a special case another type?
 - Layout
 - Move semantics
 - `constexpr` support
 - Exception Safety
 - Fallible APIs
 - Fallible Unchecked APIs
 - Iterator invalidation
 - Freestanding
 - Return type of `push_back`
 - Summary of semantic differences with vector
- Technical specification
 - `[library.requirements.organization.headers]`
 - `[library.requirements.organization.compliance]`
 - `[container.reqmts]` General container requirements
 - `[container.requirements.sequence.reqmts]`
 - `[containers.sequences.inplace_vector.syn]` Header `<inplace_vector>` synopsis

- [containers.sequences.inplace_vector] Class template `inplace_vector`
 - [containers.sequences.inplace_vector.overview] Overview`
 - [containers.sequences.inplace_vector.cons] Constructors, copy, and assignment
 - [containers.sequences.inplace_vector.destructor] Destruction
 - [containers.sequences.inplace_vector.capacity] Size and capacity
 - [containers.sequences.inplace_vector.data]
 - [containers.sequences.inplace_vector.modifiers]
 - [containers.sequences.inplace_vector.erasure] Erasure
 - [containers.sequences.inplace_vector.special] Specialized algorithms
- Acknowledgments

Changelog

- **Revision 7** Varna 2023 draft
 - ☒ Rename `static_vector` to `inplace_vector` throughout.
 - ☒ Update `try_push_back` APIs to return `T*` with rationale.
 - ☒ Update `push_back` to throw `std::bad_alloc` with rationale .
 - ☒ Trivially-copyable if `value_type` is trivially-copyable.
 - ☒ Request LEWG poll regarding `<vector>` or `<inplace_vector>` header.
 - ☒ Make `push_back` return a reference
 - ☐ Update `push_back_unchecked` with benchmark data.
 - ☐ Add a specialization of `hash` like for `std::vector` .
 - ☐ Update `operator!=` to `operator<=>` using hidden friends for them.
 - ☐ Add deduction guides (there is only one, do we need more).
 - ☐ Update `constexpr` rationale to C++23.
- **Revision 6:** for Varna 2023 following Kona's 2022 guidance
 - Updated `push_back` semantics to follow `std::vector` (note about exception to throw).
 - Added `try_push_back` returning an `optional`
 - Added `push_back_unchecked` : excedding capacity exhibits undefined behavior.
 - Added note about naming.
- **Revision 5:**
 - Update contact wording and contact data.
 - Removed naming discussion, since it was resolved (last available in P0843r4 (<https://wg21.link.p0843r4>)).
 - Removed future extensions discussion (last available in P0843r4 (<https://wg21.link.p0843r4>)).

- Addressed LEWG feedback regarding move-semantics and exception-safety.
- **Revision 4:**
 - LEWG suggested that `push_back` should be UB when the capacity is exceeded
 - LEWG suggested that this should be a free-standing header
- **Revision 3:**
 - Include LWG design questions for LEWG.
 - Incorporates LWG feedback.
- **Revision 2**
 - Replace the placeholder name `fixed_capacity_vector` with `static_vector`
 - Remove at checked element access member function.
 - Add changelog section.
- **Revision 1**
 - Minor style changes and bugfixes.

Introduction

This paper proposes a modernized `boost::static_vector<T, Capacity>` [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html). That is, a dynamically-resizable vector with compile-time fixed capacity and contiguous embedded storage in which the elements are stored within the vector object itself.

Its API closely resembles that of `std::vector<T, A>`. It is a contiguous container with $O(1)$ insertion and removal of elements at the end (non-amortized) and worst case $O(\text{size}())$ insertion and removal otherwise. Like `std::vector`, the elements are initialized on insertion and destroyed on removal. For trivial `value_type`s, the vector is fully usable inside `constexpr` functions.

Motivation and Scope

The `inplace_vector` container is useful when:

- memory allocation is not possible, e.g., embedded environments without a free store, where only a stack and the static memory segment are available,
- memory allocation imposes an unacceptable performance penalty, e.g., with respect to latency,
- allocation of objects with complex lifetimes in the *static*-memory segment is required,
- `std::array` is not an option, e.g., if non-default constructible objects must be stored,

- a dynamically-resizable array is required within `constexpr` functions,
- the storage location of the `inplace_vector` elements is required to be within the `inplace_vector` object itself (e.g. to support `memcpy` for serialization purposes).

Existing practice

There are at least 3 widely used implementations of `inplace_vector` : Boost.Container [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html), EASTL [2] (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71), and Folly [3] (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md). Boost.Container implements `inplace_vector` as a standalone type with its own guarantees. EASTL (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71) and Folly (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md) implement it via an extra template parameter in their `small_vector` types.

Custom allocators like Howard Hinnant's `stack_alloc` [4] (https://howardhinnant.github.io/stack_alloc.html) emulate `inplace_vector` on top of `std::vector` , but as discussed in the next sections, this emulation is not great.

Other prior art includes:

- P0494R0: `contiguous_container` proposal [5] (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0494r0.pdf>): proposes a `Storage` concept.
- P0597R0: `std::constexpr_vector<T>` [6] (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0597r0.html>): proposes a vector that can only be used in `constexpr` contexts.

A reference implementation of this proposal is available here (godbolt) (<https://godbolt.org/z/Pv8894xx6>).

Design

Standalone or a special case another type?

The EASTL (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71) [2] and Folly (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md) [3] special case `small_vector` , e.g., using a 4th template parameter, to make it become a `inplace_vector` . P0639R0: *Changing attack vector of the `constexpr_vector`* [7] (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0639r0.html>) proposes improving the `Allocator` concepts to

allow implementing `inplace_vector` as a special case of `vector` with a custom allocator. Both approaches produce specializations of `small_vector` or `vector` whose methods subtly differ in terms of effects, exception-safety, iterator invalidation, and complexity guarantees.

This proposal closely follows `boost::container::static_vector<T, Capacity>` [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html) and proposes `inplace_vector` as a standalone type.

Where possible, this proposal defines the semantics of `inplace_vector` to match `vector`. Providing the same programming model makes this type easier to teach, use, and makes it easy to “just change” one type on a program to, e.g., perform a performance experiment without accidentally introducing undefined behavior.

Layout

`inplace_vector` models `ContiguousContainer`. Its elements are stored and properly aligned within the `inplace_vector` object itself. If the `Capacity` is zero the container has zero size:

```
static_assert(is_empty_v<inplace_vector<T, 0>>); // for all T
```

The offset of the first element within `inplace_vector` is unspecified, and `T`s are not allowed to overlap.

The layout differs from `vector`, since `inplace_vector` does not store the `capacity` field (it’s known from the template parameter).

If `T` is trivially-copyable or `N == 0`, then `inplace_vector<T, N>` is also trivially copyable to support HPC (such as copying one between a CPU and a GPU), serialization/deserialization, which is important for use cases like sending a vector via `MPI_Send`, etc.

```
// for all C:  
static_assert(!is_trivially_copyable_v<T> || is_trivially_copyable_v<inplace_vecto
```



Move semantics

A moved-from `inplace_vector` is left in a *valid but unspecified state* (option 3 below) unless `T` is trivially-copyable, in which case the size of the `inplace_vector` does not change (array semantics, option 2 below). That is:

```
inplace_vector a(10);
inplace_vector b(std::move(a));
assert(a.size() == 10); // MAY FAIL
```

moves `a` 's elements element-wise into `b` , and afterwards the size of the moved-from `inplace_vector` may have changed.

This prevents code from relying on the size staying the same (and therefore being incompatible with changing an `inplace_vector` type back to `vector`) without incurring the cost of having to clear the `inplace_vector` .

When `T` is trivially-copyable, `array` semantics are used to provide trivial move operations.

This is different from LEWG Kona '22 Polls (<https://github.com/cplusplus/papers/issues/114#issuecomment-1312015872>) (22 in person + 8 remote) and we'd like to poll on these semantics again:

- POLL: Moving a `static_vector` should empty it (vector semantics).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
9	10	4	2	2

- POLL: Moving a `static_vector` should leave it in a valid but unspecified state.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
6	9	1	5	6

Alternatives:

1. `vector` semantics: guarantees that `inplace_vector` is left empty (this happens with move assignment when using `std::allocator<T>` and always with move construction).
 - Pro: same programming model as `vector` .
 - Pro: increases safety by requiring users to re-initialize vector elements.
 - Con: clearing an `inplace_vector` is not free.
 - Con: `inplace_vector<T, N>` can no longer be made trivially copyable for a trivially copyable `T` , as the move operations can no longer be trivial.
2. `array` semantics: guarantees that `size()` of `inplace_vector` does not change, and that elements are left in their moved-from state.
 - Pro: no additional run-time cost incurred.
 - Con: different programming model than `vector` .

3. “valid but unspecified state”

- Con: different programming model than `vector` and `array`, requires calling `size()`
- Pro: code calling `size()` is correct for both `vector` and `inplace_vector`, enabling changing the type back and forth.

constexpr support

The API of `inplace_vector<T, Capacity>` can be used in `constexpr` -contexts if `is_trivially_copyable_v<T>`, `is_default_constructible_v<T>`, and `is_trivially_destructible<T>` are true.

The implementation cost of this is small. The prototype implementation specializes the storage to use a C array with value-initialized elements.

This negatively impacts the algorithmic complexity of `inplace_vector` constructors for these types from $O(\text{size})$ to $O(\text{capacity})$. When value-initialization takes place at run-time, this difference is significant.

Vectors with large capacity requirements are better served by `vector` instead.

Exception Safety

When using the `inplace_vector` APIs, the following types of failures are expected:

1. The `value_type`’s constructors/assignment/destructors/swap can potentially throw (depends on `noexcept`),
2. Mutating operations exceeding the capacity (`push_back`, `insert`, `emplace`, `inplace_vector(value_type, size)`, `inplace_vector(begin, end) ...`), and
3. Out-of-bounds unchecked access: `front` / `back` / `pop_back` when empty, `operator[]`.

For `inplace_vector`, we choose the same semantics as for `vector`:

1. When `value_type`’s operations are invoked, `inplace_vector` exception-safety guarantees depend on whether these operations throw, which is detected with `noexcept` if the API has a narrow contract.
2. Mutating operations that exceed the capacity ask “allocator embedded within the `inplace_vector`”, which has run out of space, for more memory, and therefore throw `std::bad_alloc` like `vector` APIs do (e.g. a `pmr` “stack allocator” throws `std::bad_alloc` as well). This preserves the programming model from `vector`. `std::bad_alloc` also has the property that it never does an allocation itself.

3. Out-of-bounds unchecked access is a precondition violation.

Alternatives:

1. Throw `std::bad_alloc` when the `inplace_vector` is out-of-memory:
 - **Pros:** same programming model as `vector` .
2. Throw “some other exception” when the `inplace_vector` is out-of-memory:
 - **Pros:** to be determined.
 - **Cons:** different programming model as `vector` .
3. Abort the process
 - **Pros:** portability to embedded platforms without exception support
 - **Cons:** different programming model than `vector`
4. Precondition violation
 - **Cons:** different programming model than `vector` , users responsible for checking before modifying vector size, etc.

Fallible APIs

We add the following new fallible APIs which, when the vector size equal its capacity, return `nullptr` (do not throw `bad_alloc`) without moving from the inputs, enabling them to be re-used:

```
constexpr T* inplace_vector<T, C>::try_push_back(const T& value);  
constexpr T* inplace_vector<T, C>::try_push_back(T&& value);  
  
template<class... Args>  
constexpr T* try_emplace_back(Args&&... args);
```

They can be used as follows

```
T value = T();  
if (!v.try_push_back(value)) {  
    std::cerr << "Failed to insert " << value << std::endl; // value not moved  
    std::terminate();  
}
```


Fallible Unchecked APIs

We add the following new fallible unchecked APIs for which exceeding the capacity is a precondition violation:

```
constexpr T& inplace_vector<T, C>::push_back_unchecked(const T& value);  
constexpr T& inplace_vector<T, C>::push_back_unchecked(T&& value);  
  
template<class... Args>  
constexpr T& emplace_back_unchecked(Args&&... args);
```

These APIs were requested in LEWG Kona '22 (<https://github.com/cplusplus/papers/issues/114#issuecomment-1312015872>) (22 in person + 8 remote):

- POLL: If `static_vector` has unchecked operations (e.g. `push_back_unchecked`), it is okay for checked operations (e.g. `push_back`) to throw when they run out of space.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
14	4	2	4	1

The potential impact of the three APIs on code generation is shown here (<https://clang.godbolt.org/z/MbG17q8x1>), where the main difference between `try_push_back` and `push_back_unchecked` is the presence of an extra branch in `try_push_back`.

The authors have made several attempts at micro-benchmarking the performance difference between `push_back_unchecked` and `try_push_back`, and the results will be presented to LEWG.

Iterator invalidation

`inplace_vector` iterator invalidation guarantees differ from `std::vector`:

- moving a `inplace_vector` invalidates all iterators, and
- swapping two `inplace_vector`s invalidates all iterators.

`inplace_vector` APIs that potentially invalidate iterators are: `resize(n)`, `resize(n, v)`, `pop_back`, `erase`, and `swap`.

Freestanding

The `inplace_vector` APIs are all freestanding.

LWG asked for `inplace_vector` to be part of the `<vector>` header.

Request LEWG to poll on that.

Return type of `push_back`

In C++20, both `push_back` and `emplace_back` were slated to return a reference (they used to both return `void`). Even with plenary approval, changing `push_back` turned out to be an ABI break that was backed out, leaving the situation where `emplace_back` returns a reference but `push_back` is still `void`. This ABI issue doesn't apply to new types. Should `push_back` return a reference to be consistent with `emplace_back`, or should it be consistent with older containers?

Request LEWG to poll on that.

Summary of semantic differences with `vector`

Aspect	<code>vector</code>	<code>inplace_vector</code>
Capacity	Indefinite	N
Move and swap	O(1), no iterators invalidated	array semantics: O(size), invalidates all iterators
Moved from	left empty (this happens with move assignment when using <code>std::allocator<T></code> and always with move construction)	valid but unspecified state except if T is trivially-copyable, in which case array semantics
Default construction and destruction of trivial types	O(1)	O(capacity)
Is empty when zero capacity?	No	Yes
Trivially-copyable if <code>is_trivially_copyable_v<T></code> ?	No	Yes

Technical specification

Note to editor: This enhancement is a pure header-only addition to the C++ standard library as the *freestanding* `<inplace_vector>` header. It belongs in the “Sequence containers” ([\ref{sequences}](#)) part of the “Containers library” ([\ref{containers}](#)) as “Class template `inplace_vector`”.

- **UNRESOLVED QUESTION:** LWG recommends making this header part of `<vector>` .

Note: one of the primary use cases for this container is embedded/freestanding applications. Do we impact those by not putting it in a free-standing header and adding it to `<vector>` instead?

[library.requirements.organization.headers]

Add `<inplace_vector>` to [tab:headers.cpp].

[library.requirements.organization.compliance]

Add `<inplace_vector>` tp [tab:headers.cpp.fs]:

- [containers]
- Subclause: containers
- Headers: `<inplace_vector>`

[container.reqmts] General container requirements

<http://eel.is/c++draft/container.reqmts> (<http://eel.is/c++draft/container.reqmts>)

1. A type `X` meets the container requirements if the following types, statements, and expressions are well-formed and have the specified semantics.

`typename X::value_type`

- Result: `T`
- Preconditions: `T` is `Cpp17Erasable` from `X` (see [container.alloc.reqmts], below).

typename X::reference

- Result: T&

typename X::const_reference

- Result: const T&

typename X::iterator

- Result: A type that meets the forward iterator requirements ([forward.iterators]) with value type T . The type `X::iterator` is convertible to `X::const_iterator`.

typename X::const_iterator

- Result: A type that meets the requirements of a constant iterator and those of a forward iterator with value type T .

typename X::difference_type

- Result: A signed integer type, identical to the difference type of `X::iterator` and `X::const_iterator`.

typename X::size_type

- Result: An unsigned integer type that can represent any non-negative value of `X::difference_type`.

X u;

X u = X();

- Postconditions: `u.empty()`
- Complexity: Constant.

X u(a);

X u = a;

- Preconditions: `T` is `Cpp17CopyInsertable` into `X` (see below).
- Postconditions: `u == a`
- Complexity: Linear.

```
X u(rv);
X u = rv;
```

- Postconditions: `u` is equal to the value that `rv` had before this construction.
- Complexity: Linear for array and inplace_vector and constant for all other standard containers.

```
a = rv
```

- Result: `X&` .
- Effects: All existing elements of `a` are either move assigned to or destroyed.
- Postconditions: If `a` and `rv` do not refer to the same object, `a` is equal to the value that `rv` had before this assignment.
- Complexity: Linear.

```
a.~X()
```

- Result: `void`
- Effects: Destroys every element of `a` ; any memory obtained is deallocated.
- Complexity: Linear.

```
a.begin()
```

- Result: `iterator` ; `const_iterator` for constant `a` .
- Returns: An iterator referring to the first element in the container.
- Complexity: Constant.

```
a.end()
```

- Result: `iterator` ; `const_iterator` for constant `a` .
- Returns: An iterator which is the past-the-end value for the container.
- Complexity: Constant.

`a.cbegin()`

- Result: `const_iterator` .
- Returns: `const_cast<X const&>(a).begin()`
- Complexity: Constant.

`a.cend()`

- Result: `const_iterator` .
- Returns: `const_cast<X const&>(a).end()`
- Complexity: Constant.

`i <=> j`

- Result: `strong_ordering` .
- Constraints: `X::iterator` meets the random access iterator requirements.
- Complexity: Constant.

`a == b`

- Preconditions: `T` meets the `Cpp17EqualityComparable` requirements.
- Result: Convertible to `bool` .
- Returns: `equal(a.begin(), a.end(), b.begin(), b.end())` [Note 1: The algorithm `equal` is defined in [alg.equal]. — end note]
- Complexity: Constant if `a.size() != b.size()` , linear otherwise.
- Remarks: `==` is an equivalence relation.

`a != b`

- Effects: Equivalent to `!(a == b)` .

`a.swap(b)`

- Result: `void`
- Effects: Exchanges the contents of `a` and `b` .

- Complexity: Linear for array and inplace_vector, and constant for all other standard containers.

`swap(a, b)`

- Effects: Equivalent to `a.swap(b)` .

`r = a`

- Result: `x&` .
- Postconditions: `r == a` .
- Complexity: Linear.

`a.size()`

- Result: `size_type` .
- Returns: `distance(a.begin(), a.end())` , i.e. the number of elements in the container.
- Complexity: Constant.
- Remarks: The number of elements is defined by the rules of constructors, inserts, and erases.

`a.max_size()`

- Result: `size_type` .
- Returns: `distance(begin(), end())` for the largest possible container.
- Complexity: Constant.

`a.empty()`

- Result: Convertible to `bool` .
- Returns: `a.begin() == a.end()`
- Complexity: Constant.
- Remarks: If the container is empty, then `a.empty()` is true.

1. In the expressions

```

i == j
i != j
i < j
i <= j
i >= j
i > j
i <=> j
i - j

```

where `i` and `j` denote objects of a container's iterator type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

Unless otherwise specified, all containers defined in this Clause obtain memory using an allocator (see [allocator.requirements]).

[Note 2: In particular, containers and iterators do not store references to allocated elements other than through the allocator's pointer type, i.e., as objects of type `P` or `pointer_traits<P>::template rebind<unspecified>`, where `P` is `allocator_traits<allocator_type>::pointer`. — end note]

Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. Such move construction of the allocator shall not exit via an exception. All other constructors for these container types take a `const allocator_type&` argument.

[Note 3: If an invocation of a constructor uses the default value of an optional allocator argument, then the allocator type must support value-initialization. — end note]

A copy of this allocator is used for any memory allocation and element construction performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if

1. `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value` ,
2. `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value` , or

3. `allocator_traits<allocator_type>::propagate_on_container_swap::value` is `true` within the implementation of the corresponding container operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement.
4. The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than `array` and `inplace_vector`, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Lvalues of any Compare, Pred, or Hash types belonging to `a` and `b` shall be swappable and shall be exchanged by calling `swap` as described in [swappable.requirements]. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is `true`, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in [swappable.requirements]. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.

[container.requirements.sequence.reqmts]

<http://eel.is/c++draft/sequence.reqmts> (<http://eel.is/c++draft/sequence.reqmts>)

`sequence.reqmts.1` : The library provides ~~four~~the following basic kinds of sequence containers: `vector`, `inplace_vector`, `forward_list`, `list`, and `deque`.

`sequence.reqmts.2` : [...] `vector` is the type of sequence container that should be used by default. `inplace_vector` should be used when the container has a fixed capacity known during translation. `array` should be used when the container has a fixed size known during translation.

[...]

The following operations are provided for some types of sequence containers but not others. An implementation shall implement them so as to take amortized constant time.

`a.front()`

- Result: reference ; `const_reference` for constant `a`.
- Returns: `*a.begin()`

- Remarks: Required for `basic_string` , `array` , `deque` , `forward_list` , `list` , `inplace_vector` , and `vector`.

`a.back()`

- Result: reference ; `const_reference` for constant `a` .
- Effects: Equivalent to:

```
auto tmp = a.end();
--tmp;
return *tmp;
```

- Remarks: Required for `basic_string` , `array` , `deque` , `list` , `inplace_vector` , and `vector`.

`a.emplace_front(args)`

- Result: reference
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `args` .
- Effects: Prepends an object of type `T` constructed with `std::forward<Args>(args)...`.
- Returns: `a.front()`.
- Remarks: Required for `deque` , `forward_list` , and `list` .

`a.emplace_back(args)`

- Result: reference
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `args` . For `vector` and `inplace_vector` , `T` is also `Cpp17MoveInsertable` into `X` .
- Effects: Appends an object of type `T` constructed with `std::forward<Args>(args)...` .
- Returns: `a.back()`.
- Remarks: Required for `deque` , `list` , `inplace_vector` , and `vector`.

`a.push_front(t)`

- Result: `void`
- Preconditions: `T` is `Cpp17CopyInsertable` into `X` .
- Effects: Prepends a copy of `t` .

- Remarks: Required for `deque` , `forward_list` , and `list` .

`a.push_front(rv)`

- Result: `void`
- Preconditions: `T` is `Cpp17MoveInsertable` into `X` .
- Effects: Prepends a copy of `rv` .
- Remarks: Required for `deque` , `forward_list` , and `list` .

`a.prepend_range(rg)`

- Result: `void`
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `*ranges::begin(rg)` .
- Effects: Inserts copies of elements in `rg` before `begin()` . Each iterator in the range `rg` is dereferenced exactly once. [Note 3: The order of elements in `rg` is not reversed. — end note]
- Remarks: Required for `deque` , `forward_list` , and `list` .

`a.push_back(t)`

- Result: Reference
- Returns: `a.back()` .
- Preconditions: `T` is `Cpp17CopyInsertable` into `X` .
- Effects: Appends a copy of `t` .
- Remarks: Required for `basic_string` , `deque` , `list` , `inplace_vector` , and `vector` .

`a.push_back(rv)`

- Result: Reference
- Returns: `a.back()`
- Preconditions: `T` is `Cpp17MoveInsertable` into `X` .
- Effects: Appends a copy of `rv` .
- Remarks: Required for `basic_string` , `deque` , `list` , `inplace_vector` , and `vector` .

`a.append_range(rg)`

- Result: `void`
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `*ranges::begin(rg)`. For `vector` and `inplace_vector`, `T` is also `Cpp17MoveInsertable` into `X`.
- Effects: Inserts copies of elements in `rg` before `end()`. Each iterator in the range `rg` is dereferenced exactly once.
- Remarks: Required for `deque`, `list`, `inplace_vector`, and `vector`.

`a.pop_front()`

- Result: `void`
- Preconditions: `a.empty()` is `false`.
- Effects: Destroys the first element.
- Remarks: Required for `deque`, `forward_list`, and `list`.

`a.pop_back()`

- Result: `void`
- Preconditions: `a.empty()` is `false`.
- Effects: Destroys the last element.
- Remarks: Required for `basic_string`, `deque`, `list`, `inplace_vector`, and `vector`.

`a[n]`

- Result: `reference`; `const_reference` for constant `a`
- Returns: `*(a.begin() + n)`
- Remarks: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

`a.at(n)`

- Result: `reference`; `const_reference` for constant `a`
- Returns: `*(a.begin() + n)`
- Throws: `out_of_range` if `n >= a.size()`.
- Remarks: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

<inplace_vector> synopsis

```
namespace std {

template <typename T, size_t N>
class inplace_vector {
public:
// types:
using value_type = T;
using pointer = T*;
using const_pointer = const T*;
using reference = value_type&;
using const_reference = const value_type&;
using size_type = size_t;
using difference_type = ptrdiff_t;
using iterator = implementation-defined; // see [container.requirements]
using const_iterator = implementation-defined; // see [container.requirements]
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;

// [containers.sequences.inplace_vector.cons]
constexpr inplace_vector() noexcept;
constexpr inplace_vector(const inplace_vector&);
constexpr inplace_vector(inplace_vector&&)
noexcept(!N || is_nothrow_move_constructible_v<value_type>);
constexpr explicit inplace_vector(size_type n);
constexpr inplace_vector(size_type n, const value_type& value);
template <class InputIterator>
constexpr inplace_vector(InputIterator first, InputIterator last);
constexpr inplace_vector(initializer_list<value_type> il);

// [containers.sequences.inplace_vector.cons]
constexpr inplace_vector& operator=(const inplace_vector& other);
constexpr inplace_vector& operator=(inplace_vector&& other)
noexcept(!N || is_nothrow_move_assignable_v<value_type>);
template <class InputIterator>
constexpr void assign(InputIterator first, InputIterator last);
constexpr void assign(size_type n, const value_type& u);
constexpr void assign(initializer_list<value_type> il);

// [containers.sequences.inplace_vector.destructor]
constexpr ~inplace_vector();

// iterators
constexpr iterator          begin()          noexcept;
constexpr const_iterator    begin()          const noexcept;
constexpr iterator          end()            noexcept;
constexpr const_iterator    end()            const noexcept;
```

```

constexpr reverse_iterator      rbegin()          noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator      rend()            noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_iterator        cbegin() const noexcept;
constexpr const_iterator        cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// [containers.sequences.inplace_vector.members] size/capacity
[[nodiscard]] constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
static constexpr size_type max_size() noexcept;
static constexpr size_type capacity() noexcept;
constexpr void resize(size_type sz);
constexpr void resize(size_type sz, const value_type& c);

// [containers.sequences.inplace_vector.members] element and data access:
constexpr reference      operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference      front();
constexpr const_reference front() const;
constexpr reference      back();
constexpr const_reference back() const;
constexpr T* data()      noexcept;
constexpr const T* data() const noexcept;

// 5.7, modifiers:
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
constexpr iterator insert(const_iterator position, size_type n, const value_type&
template <class InputIterator>
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
constexpr iterator insert(const_iterator position, initializer_list<value_type> il);

template <class... Args>
    constexpr iterator emplace(const_iterator position, Args&&... args);
template <class... Args>
    constexpr reference emplace_back(Args&&... args);
constexpr T& push_back(const value_type& x);
constexpr T& push_back(value_type&& x);

constexpr void pop_back();
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void clear() noexcept;

constexpr void swap(inplace_vector& x)

```

```

    noexcept(is_nothrow_swappable_v<value_type> &&
             is_nothrow_move_constructible_v<value_type>);
};

template<class InputIterator>
inplace_vector(InputIterator, InputIterator)
-> inplace_vector<iter-value-type<InputIterator>>;

template <typename T, size_t N>
constexpr bool operator==(const inplace_vector<T, N>& a, const inplace_vector<T, N>
template <typename T, size_t N>
constexpr bool operator!=(const inplace_vector<T, N>& a, const inplace_vector<T, N>
template <typename T, size_t N>
constexpr bool operator<(const inplace_vector<T, N>& a, const inplace_vector<T, N>
template <typename T, size_t N>
constexpr bool operator<=(const inplace_vector<T, N>& a, const inplace_vector<T, N>
template <typename T, size_t N>
constexpr bool operator>(const inplace_vector<T, N>& a, const inplace_vector<T, N>
template <typename T, size_t N>
constexpr bool operator>=(const inplace_vector<T, N>& a, const inplace_vector<T, N>

// 5.8, specialized algorithms:
template <typename T, size_t N>
constexpr void swap(inplace_vector<T, N>& x, inplace_vector<T, N>& y)
    noexcept(noexcept(x.swap(y)));

} // namespace std

```

[containers.sequences.inplace_vector] Class template inplace_vector

[containers.sequences.inplace_vector.overview] Overview`

1. A `inplace_vector` is a contiguous container that supports constant time insert and erase operations at the end; insert and erase in the middle take linear time. Its capacity is part of its type and its elements are stored within the `inplace_vector` object itself, meaning that that if `v` is a `inplace_vector<T, N>` then it obeys the identity `&v[n] == &v[0] + n` for all $0 \leq n \leq v.size()$.
2. A `inplace_vector` satisfies the container requirements (`\ref{container.requirements}`) with the exception of the `swap` member function, whose complexity is linear instead of constant. It satisfies the sequence container requirements, including the optional sequence container requirements (`\ref{sequence.reqmts}`), with the exception of the `push_front`, `pop_front`, and

`emplace_front` member functions, which are not provided. It satisfies the reversible container ([`container.requirements`]) and contiguous container ([`container.requirements.general`]) requirements. Descriptions are provided here only for operations on `inplace_vector` that are not described in one of these tables or for operations where there is additional semantic information.

3. Class `inplace_vector` relies on the implicitly-declared special member functions [`class.default.ctor`], [`class.dtor`], and [`class.copy.ctor`] to conform to the container requirements table in [`container.requirements`]. In addition to the requirements specified in the container requirements table, the move constructor and move assignment operator for array require that `T` be `Cpp17MoveConstructible` or `Cpp17MoveAssignable`, respectively.
4. The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements ([`iterator.requirements.general`])."

[`containers.sequences.inplace_vector.cons`] Constructors, copy, and assignment

```
constexpr inplace_vector() noexcept;
```

- *Effects*: Constructs an empty `inplace_vector`.
- *Postconditions*: `empty()` is true.
- *Complexity*: Constant.

```
constexpr inplace_vector(inplace_vector&& rv);
```

- *Effects*: Constructs a `inplace_vector` by move-inserting the elements of `rv`.
- *Mandates*: `is_move_constructible_v<value_type>` is true.
- *Postconditions*: The `inplace_vector` is equal to the value that `rv` had before this construction.
- *Complexity*: Linear in `rv.size()`.

```
constexpr explicit inplace_vector(size_type n);
```

- *Effects*: Constructs a `inplace_vector` with `n` default-inserted elements.

- *Mandates*: `is_default_constructible_v<value_type>` is true.
- *Preconditions*: `n <= capacity()` is true.
- *Complexity*: Linear in `n`.

```
constexpr inplace_vector(size_type n, const value_type& value);
```

- *Effects*: Constructs a `inplace_vector` with `n` copies of `value`.
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Preconditions*: `n <= capacity()` is true.
- *Complexity*: Linear in `n`.

```
template <class InputIterator>
constexpr inplace_vector(InputIterator first, InputIterator last);
```

- *Effects*: Constructs a `inplace_vector` equal to the range `[first, last)`.
- *Mandates*: `is_constructible_v<value_type, decltype(*first)>` is true.
- *Preconditions*: `distance(first, last) <= capacity()` is true.
- *Complexity*: Linear in `distance(first, last)`.

[containers.sequences.inplace_vector.destructor] Destruction

```
constexpr ~inplace_vector();
```

- *Effects*: Destroys the `inplace_vector` and its elements.
- *Remarks*: This destructor is trivial if the destructor of `value_type` is trivial.

[containers.sequences.inplace_vector.capacity] Size and capacity

```
static constexpr size_type capacity() noexcept
static constexpr size_type max_size() noexcept
```

- *Returns*: `N`.

```
constexpr void resize(size_type sz);
```

- *Effects*: If `sz < size()` , erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-constructed elements.
- *Mandates*: `is_default_constructible_v<value_type>` is true.
- *Preconditions*: `sz <= capacity()` is true.
- *Complexity*: Linear in `sz` .

```
constexpr void resize(size_type sz, const value_type& c);
```

- *Effects*: If `sz < size()` , erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` .
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Preconditions*: `sz <= capacity()` is true.
- *Complexity*: Linear in `sz` .

[containers.sequences.inplace_vector.data]

Element and data access

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

- *Returns*: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `inplace_vector` , `data() == addressof(front())` .
- *Complexity*: Constant time.

[containers.sequences.inplace_vector.modifiers]

Modifiers

Note to LWG: All modifiers have a pre-condition on not exceeding the `capacity()` when inserting elements. That is, exceeding the `capacity()` of the vector is undefined behavior. This supports some of the major use cases of this container (embedded, freestanding, etc.) and was required by the stakeholders during LEWG review. Currently, this

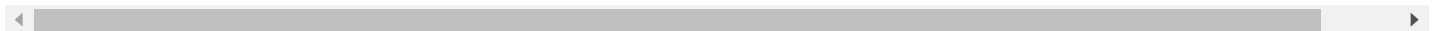
provides maximum freedom to the implementation to choose an appropriate behavior: `abort` , `assert` , throw an exception (which exception? `bad_alloc` ? `std::length_error` ? `logic_error` ? `out_of_bounds` ? etc.). In the future, this freedom allows us to specify these pre-conditions using contracts.

Note to LWG: Because all modifiers have preconditions, they all have narrow contracts and are not unconditionally `noexcept` .

```
constexpr iterator insert(const_iterator position, const value_type& x);
```

- *Effects*: Inserts `x` at `position` and invalidates all references to elements after `position` .
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Complexity*: Linear in `size()` .
- *Remarks*: If an exception is thrown by `value_type` 's copy constructor and `is_nothrow_move_constructible_v<value_type>` is true there are no effects. Otherwise, if an exception is thrown by `value_type` 's copy constructor the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, size_type n, const value_type&
```



- *Effects*: Inserts `n` copies of `x` at `position` and invalidates all references to elements after `position` .
- *Preconditions*: `n <= capacity() - size()` is true.
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Complexity*: Linear in `size()` and `n` .
- *Remarks*: If an exception is thrown by `value_type` 's copy constructor and `is_nothrow_move_constructible_v<value_type>` is true there are no effects. Otherwise, if an exception is thrown by `value_type` 's copy constructor the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, value_type&& x);
```

- *Effects*: Inserts `x` at `position` and invalidates all references to elements after `position`.
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_move_constructible_v<value_type>` is true.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
template <typename InputIterator>
constexpr iterator insert(const_iterator position, InputIterator first, InputItera
```

- *Effects*: Inserts elements in range `[first, last)` at `position` and invalidates all references to elements after `position`.
- *Preconditions*: `distance(first, last) <= capacity() - size()` is true.
- *Mandates*: `is_constructible_v<value_type, decltype(*first)>` is true.
- *Complexity*: Linear in `size()` and `distance(first, last)`.
- *Remarks*: If an exception is thrown by `value_type` constructor from `decltype(*first)` and `is_nothrow_move_constructible_v<value_type>` is true there are no effects. Otherwise, if an exception is thrown by `value_type`'s constructor from `decltype(*first)` the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, initializer_list<value_type> il
```

- *Effects*: Inserts elements of `il` at `position` and invalidates all references to elements after `position`.
- *Preconditions*: `il.size() <= capacity() - size()` is true.
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Complexity*: Linear in `size()` and `il.size()`.
- *Remarks*: If an exception is thrown by `value_type`'s copy constructor and `is_nothrow_move_constructible_v<value_type>` is true there are no effects. Otherwise, if an exception is thrown by `value_type`'s copy constructor the effects are *unspecified*.

```
template <class... Args>
constexpr iterator.emplace(const_iterator position, Args&&... args);
```

- *Effects*: Inserts an element constructed from `args...` at `position` and invalidates all references to elements after `position`.
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_constructible_v<value_type, Args...>` is true.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s constructor from `args...` and `is_nothrow_move_constructible_v<value_type>` is true there are no effects. Otherwise, if an exception is thrown by `value_type`'s constructor from `args...` the effects are *unspecified*.

```
template <class... Args>
constexpr reference.emplace_back(Args&&... args);
```

- *Effects*: Inserts an element constructed from `args...` at the end.
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_constructible_v<value_type, Args...>` is true.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s constructor from `args...` there are no effects.

```
constexpr reference.push_back(const value_type& x);
```

- *Effects*: Inserts a copy of `x` at the end.
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_copy_constructible_v<value_type>` is true.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s copy constructor there are no effects.

```
constexpr reference push_back(value_type&& x);
```

- *Effects*: Moves `x` to the end.
- *Preconditions*: `size() < capacity()` is true.
- *Mandates*: `is_move_constructible_v<value_type>` is true.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor there are no effects.

```
constexpr void pop_back();
```

- *Effects*: Removes the last element of the container and destroys it.
- *Preconditions*: `!empty()` is true.
- *Complexity*: Constant.

[containers.sequences.inplace_vector.erase] Erasure

```
constexpr iterator erase(const_iterator position);
```

- *Effects*: Removes the element at `position`, destroys it, and invalidates references to elements after `position`.
- *Preconditions*: `position` in range `[begin(), end())` is true.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
constexpr iterator erase(const_iterator first, const_iterator last);
```

- *Effects*: Removes the elements in range `[first, last)`, destroying them, and invalidating references to elements after `last`.
- *Preconditions*: `[first, last)` in range `[begin(), end())` is true.
- *Complexity*: Linear in `size()` and `distance(first, last)`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
constexpr void swap(inplace_vector& x)
    noexcept(is_nothrow_swappable_v<value_type> &&
              is_nothrow_move_constructible_v<value_type>);
```

- *Effects*: Exchanges the contents of `*this` with `x`. All references to the elements of `*this` and `x` are invalidated.
- *Complexity*: Linear in `size()` and `x.size()`.
- *Remarks*: Shall not participate in overload resolution unless `is_move_constructible_v<value_type>` is true and `is_swappable_v<value_type>` is true.

[containers.sequences.inplace_vector.special] Specialized algorithms

```
template <typename T, size_t N>
constexpr void swap(inplace_vector<T, N>& x,
                    inplace_vector<T, N>& y)
    noexcept(noexcept(x.swap(y)));
```

- *Constraints*: This function shall not participate in overload resolution unless `is_swappable_v<T>` is true.
- *Effects*: As if by `x.swap(y)`.
- *Complexity*: Linear in `size()` and `x.size()`.

Acknowledgments

This proposal is based on Boost.Container's `boost::container::static_vector`, mainly authored by Adam Wulkiewicz, Andrew Hundt, and Ion Gaztanaga. The reference implementation is based on Howard Hinnant's `std::vector` implementation in libc++ and its test-suite. The following people provided valuable feedback that influenced some aspects of this proposal: Walter Brown, Zach Laine, Rein Halbersma, Andrzej Krzemiński, Casey Carter and many others.